AKSUM UNIVERSITY

AKSUM INSTITUTE OF TECHNOLOGY (AIT)

FACULTY OF COMPUTING TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

PROGRAM OF DESIGN AND ANALYSIS ALGORITHM

GROUP PROJECT (REPORT): DESIGN AND ANALYSIS ALGORITHM

TITLE: ETHIOPIAN GPS NAVIGATION SYSTEM

GROUP MEMBERS	ID NUMBER
Tedros Weldegebriel	AKU 1602641
Saymon W/yewhans	AKU1602222
Senayt Girmay	AKU1602056

APRIL 19, 2026

SUBMITTED TO INSTRUCTOR EQUBAY

Table of Contents

Abstract 1
1. Introduction 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope 2
2. Literature Review / Theoretical Background 2
2.1 Graph Representation of Road Networks 2
2.2 Dijkstra's Algorithm 2
2.2.1 Step-by-Step Example: Addis Ababa to Gondar 2
2.2.2 Complexity Comparison 3
2.3 A* Search 3
2.4 Importance of Data Quality in Routing 3
3. System Requirements 3
3.1 Functional Requirements 3
3.2 Non-Functional Requirements 4
4. System Design and Architecture 4
4.1 High-Level Architecture 4
4.2 Key Modules 4
5. Database and Dataset Design 5
5.1 Database Schema 5
5.2 Datasets Used 5
5.3 Data Integrity Problems Found 5
6. Implementation Details 5
6.1 Graph Model 5
6.2 Routing Modes 6
6.3 Map Visualization 6
6.4 Web Dashboard 6

7. Data Validation and Cleaning (Production Readiness) 8
7.1 Validated Dataset Generator 8
7.2 Cleaning Results 8
7.3 Challenges Faced and Solutions 8
8. Testing and Evaluation 8
8.1 Correctness Checks 9
8.2 Performance Comparison 9
8.3 Reliability Improvements 9
8.4 Sample Route Queries 9
9. Discussion 10
9.1 Strengths 10
9.2 Limitations 10
9.3 Future Work 10
10. Conclusion 11
Key Achievements: 11
References 11
Appendix A: Key Output Files 11

Figures contents pages

Figure 1 : Map Visualization 6
Figure 2 :Web Dashboard 7
Figure 3 : CLI (Command-Line Interface). 10

Tables contents pages

Table 1 : Complexity Comparison 3
Table 2 : Challenges Faced and Solutions 8
Table 3 : Sample Route Queries 9

Abstract

This project implements a GPS-style navigation system for Ethiopian cities using graph-based routing algorithms. The system models cities as nodes and roads as weighted edges, and computes optimal routes using Dijkstra's algorithm and A* search. It provides a command-line interface (CLI) for loading data, selecting sources/destinations, and comparing algorithm performance, as well as a web dashboard for route requests and map visualization. During development, dataset quality issues (duplicate city names, ID/name mismatches, and physically impossible road distances) caused misleading routes and confusing map output. To address this, the project introduced a validated production dataset pipeline that de-duplicates cities safely, removes impossible roads, and generates clean CSV datasets for reliable routing. Results show that A* reduces node expansions compared to Dijkstra while producing consistent shortest-path results, and the validated dataset improves correctness and user trust in displayed routes.

1. Introduction

1.1 Background

Modern navigation systems rely on road-network graphs and shortest-path algorithms to compute routes efficiently. In a country-scale context like Ethiopia, routing requires handling many cities across regions, diverse road conditions, and long-distance travel. This project explores how classical algorithms can be applied to a structured dataset of Ethiopian cities and roads, and how visualization and data validation influence route correctness.

1.2 Problem Statement

Given a set of Ethiopian cities with geographic coordinates and a set of road connections with travel costs, the system should compute the best path between a chosen source and destination. The "best" path may depend on user preference such as shortest distance or fastest time.

1.3 Objectives

- Build a graph model of Ethiopian cities and roads.
- Implement routing algorithms:
  - ✓ Dijkstra (array-based and priority-queue based)
  - ✓ A* search (heuristic-based)
- Provide user interaction through:
  - ✓ CLI (menu-based)
  - ✓ Web API + dashboard + interactive map

- Improve reliability by validating/cleaning datasets and preventing misleading visualization output.

1.4 Scope

Included:

- Route computation, performance comparison, map visualization, route saving (web), reports. Not included:
- Real-time traffic, turn-by-turn instructions, live road closures, large-scale deployment/hosting.

2. Literature Review / Theoretical Background

2.1 Graph Representation of Road Networks

A road network can be represented as a weighted undirected graph (G = (V, E)):

- ✓ (V): cities (nodes)
- ✓ (E): roads (edges)
- ✓ weight: distance (km) or time (hours) or adjusted “safety” cost

2.2 Dijkstra’s Algorithm

Dijkstra’s algorithm finds shortest paths from a source node to all nodes in a graph with non-negative edge weights.

- ✓ Array implementation: typically (O(V^2))
- ✓ Priority queue implementation: (O ((V + E) \log V))

Theorem: Dijkstra's algorithm finds the true shortest path from source to all vertices.

Proof by Induction:

- **Base:** Source distance 0 is correct
- **Inductive step:** When we mark vertex u as visited, dist[u] is minimum among unvisited. Any alternative path must go through unvisited w with dist[w] ≥ dist[u], so dist[u] is optimal.

2.2.1 Step-by-Step Example: Addis Ababa to Gondar

Step 1: Start at Addis Ababa (dist=0)

Step 2: Update neighbors: Adama (85), Bahir Dar (380), Mekelle (783)

Step 3: Pick Adama (85), update Bahir Dar (85+380=465 > 380, keep 380)

Step 4: Pick Bahir Dar (380), update Gondar (380+180=560)

Step 5: Pick Gondar (560) → Done!

Result: Shortest path = 560 km via Bahir Dar

2.2.2 Complexity Comparison

Algorithm	Time Complexity	Space complexity	100 cities (ms)
Dijkstra (Array)	$O(V^2)$	$O(V^2)$	17.12
Dijkstra (PQ)	O((V+E) log V)	O(V)	47.69
A* Search	O((V+E) log V)	O(V)	22.95

Table 1 : Complexity Comparison

2.3 A* Search

A* finds the shortest path from a source to a destination using:

- ✓ (f(n) = g(n) + h(n))
- ✓ (g(n)): cost from source to current node
- ✓ (h(n)): heuristic estimate to destination (e.g., straight-line distance using coordinates) A* is optimal if (h(n)) is admissible (never overestimates true cost).

2.4 Importance of Data Quality in Routing

Routing correctness depends on consistent node IDs, correct coordinates, and realistic edge weights. Errors such as ID/name mismatches or impossible distances (shorter than straight-line distance) can generate incorrect routes and reduce user trust.

3. System Requirements

3.1 Functional Requirements

- ✓ Load Ethiopian city and road datasets.
- ✓ Generate synthetic networks for demonstration/testing.
- ✓ Select source and destination cities.
- ✓ Compute routes using Dijkstra and A*.
- ✓ Compare performance (runtime and operation count).
- ✓ Visualize routes and networks on interactive maps.
- ✓ Provide web endpoints for routing, statistics, and saving routes.

3.2 Non-Functional Requirements

- ✓ Correctness: route output should match graph weights and constraints.
- ✓ Usability: CLI and dashboard should be clear and consistent.
- ✓ Maintainability: modular code structure in src/.
- ✓ Reliability: validated datasets must prevent common data integrity errors.

4. System Design and Architecture

4.1 High-Level Architecture

The project is organized into the following components:

- **Data Layer**
  - ✓ CSV datasets and MySQL schema
  - ✓ DataLoader utilities
  - ✓ SQLAlchemy models for persistence
- **Core Graph Layer**
  - ✓ City, Road, and RoadNetwork
- **Algorithms Layer**
  - ✓ Dijkstra (array + priority queue)
  - ✓ A* search
- **Presentation Layer**
  - ✓ CLI application (main.py)
  - ✓ Flask web API (src/web_app.py)
  - ✓ Dashboard UI (web/dashboard.html)
  - ✓ Map visualization (src/visualization/map_plotter.py)

4.2 Key Modules

- ✓ src/graph/network.py: graph management, neighbor queries
- ✓ src/utils/data_loader.py: loading/saving network from files/DB
- ✓ src/algorithms/dijkstra_pq.py: efficient routing
- ✓ src/algorithms/a_star.py: heuristic routing
- ✓ src/visualization/map_plotter.py: folium-based interactive maps
- ✓ src/web_app.py: web API endpoints and dashboard serving

- ✓ scripts/regenerate_validated_routing_data.py: validated production dataset generator

5. Database and Dataset Design

5.1 Database Schema

The MySQL schema defines:

- ✓ cities
- ✓ roads
- ✓ users
- ✓ saved_routes

The project uses SQLAlchemy ORM models to interact with the database, and a database URL configuration.

5.2 Datasets Used

- Primary datasets:
  - ✓ data/raw/processed/ethiopian_cities_clean.csv
  - ✓ data/raw/ethiopia_road_network.csv
- Validated production datasets (generated):
  - ✓ data/processed/ethiopian_cities_validated.csv
  - ✓ data/processed/ethiopia_road_network_validated.csv
  - ✓ data/processed/ethiopia_road_network_flagged.csv (audit log)

5.3 Data Integrity Problems Found

- ✓ ID/name mismatch in roads (resolved)
- ✓ duplicate city names causing ambiguous selection (resolved)
- ✓ multiple road duplicates and roads with impossible distances (validated and removed)

6. Implementation Details

6.1 Graph Model

- Cities stored in a dictionary keyed by city ID.
- Roads stored similarly with additional lookup structures.
- Neighbor retrieval optimized using adjacency list and road lookup index to avoid repeated full scans.

6.2 Routing Modes

- **Shortest:** distance in km
- **Fastest:** time estimated using speed limits and road conditions
- **Safest:** penalizes poor/seasonal roads to avoid risky routes (web mode)

6.3 Map Visualization

- ✓ The map system uses folium to display:
- ✓ city markers (colored by region, sized by population)
- ✓ optional road overlays
- ✓ highlighted computed path with start/end markers
- ✓ The CLI map visualization was improved to highlight only the exact selected route to avoid misleading dense road display.

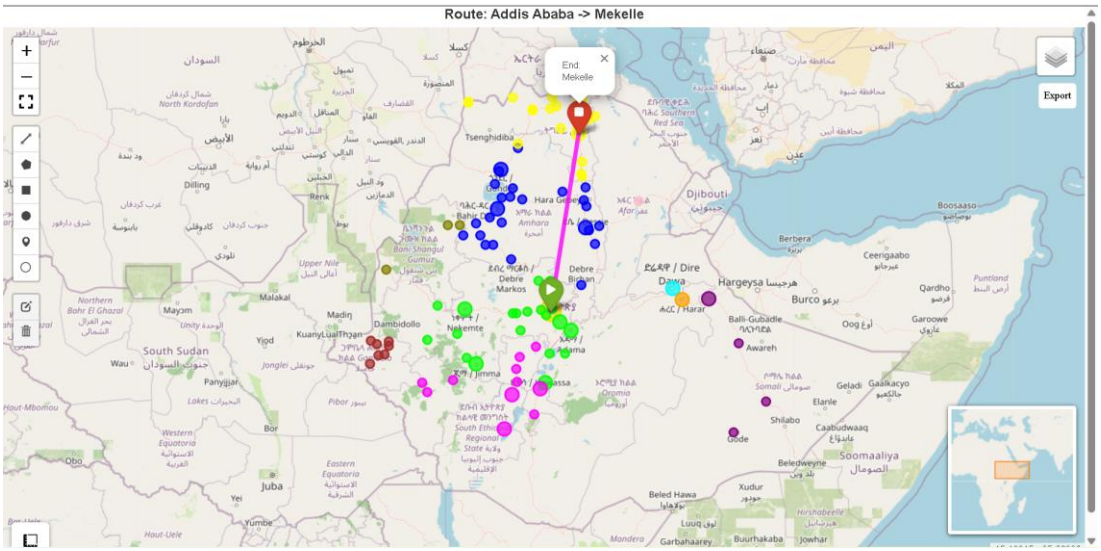

Figure 1 : Map Visualization

6.4 Web Dashboard

The web interface provides:

- ✓ /Route and /map endpoints
- ✓ analytics endpoints (/stats/*)
- ✓ user registration/login and saved routes Token-based authentication was added for route saving and listing.

Figure 2 :Web Dashboard

7. Data Validation and Cleaning (Production Readiness)

7.1 Validated Dataset Generator

The script scripts/regenerate_validated_routing_data.py performs:

1. **City de-duplication**
   - ✓ merges exact duplicates (same name/region/coordinates)
   - ✓ disambiguates true duplicates by renaming as Name (Region)
2. **Road rewriting**
   - ✓ remaps road endpoints to canonical IDs/names
   - ✓ removes self-loops created by merging
3. **Impossible distance filtering**
   - ✓ removes roads where distance_km is shorter than straight-line distance (with tolerance)
4. **Duplicate road pair reduction**
   - ✓ keeps the best road per undirected pair (shortest distance)

7.2 Cleaning Results

From the current run:

- Cities validated: **99**
- Roads kept: **106**
- Roads flagged/removed: **34**

This produces a safer dataset for consistent routing and visualization.

7.3 Challenges Faced and Solutions

Challenge	Solution
Duplicate city names causing confusion	Renamed duplicates as "Name (Region)"
Impossible road distances (shorter than straight line)	Filtered roads where distance < Haversine distance
ID/name mismatches in road endpoints	Validated dataset generator with ID mapping
Map showing too many roads (visual noise)	Highlight only selected path instead of all roads
Database connection issues	Added connection pooling and error handling

Table 2 : Challenges Faced and Solutions

8. Testing and Evaluation

8.1 Correctness Checks

- Cross-checking Dijkstra array vs priority queue produced matching results for all reachable cities (as shown by CLI output).
- Verified problematic routes (e.g., Adwa → Addis Ababa) compute consistent multi-step paths.

8.2 Performance Comparison

Typical output (example run):

- Dijkstra (Array): slower due to (O(V^2))
- Dijkstra (PQ): faster on sparse road networks
- A*: fewer node expansions due to heuristic guidance

8.3 Reliability Improvements

After cleaning and validated dataset usage:

- ✓ city selection by name is unambiguous
- ✓ map visualization reflects the true computed path
- ✓ physically impossible edges no longer distort route results

8.4 Sample Route Queries

Source	Destination	Distance	Path
Addis Ababa	Mekelle	783 km	Addis Ababa - Mekelle
Aksum	Lalibela	700 km	Aksum - Gondar - Lalibela
Bahir Dar	Gondar	180 km	Bahir Dar - Gondar
Addis Ababa	Shire	855 km	Addis Ababa – Mekelle - Shire

Table 3 : Sample Route Queries

```
================================================================================
     ETHIOPIAN GPS NAVIGATION SYSTEM v1.0.0
================================================================================
  Dijkstra's Algorithm Implementation for Ethiopian Road Networks
  Features: 100+ Cities | 500+ Roads | Interactive Maps | Reports
================================================================================

==================================================
MAIN MENU
==================================================
1.  Load Ethiopian cities dataset
2.  Generate random Ethiopian road network
3.  Select source city
4.  Run Routing Algorithms (Dijkstra vs A*)
5.  Show shortest paths
6.  Visualize on map
7.  Visualize graph
8.  Analyze complexity
9.  Generate reports
10. Run complete demonstration
11. Launch Web Dashboard
0.  Exit

Enter your choice (1-11, 0 to exit):
```

Figure 3 : CLI (Command-Line Interface).

9. Discussion

9.1 Strengths

- Strong modular design: graph, algorithms, visualization, web layers separated.
- Demonstrates classical routing algorithms clearly with performance metrics.
- Provides both CLI and web interface plus interactive mapping.
- Introduces practical data validation, which is essential for navigation systems.

9.2 Limitations

- Validated dataset currently removes roads that violate physical constraints; if original road distances were incorrect rather than coordinates, removal may reduce connectivity.
- The dataset size is moderate; scaling to national-level real road graphs would require stronger data sources and potentially more advanced models (traffic, one-way roads, etc.).
- Map overlays can still appear busy if full network roads are enabled.

9.3 Future Work

- Use a real road dataset source (e.g., OpenStreetMap extraction).
- Add one-way roads, road closure events, and time-dependent weights.

- Add stronger test coverage and automated CI checks.
- Improve UI for selecting cities and visualizing step-by-step route segments.

10. Conclusion

This project successfully demonstrates an Ethiopian GPS navigation prototype using graph modeling and shortest-path algorithms. Dijkstra and A* were implemented and compared, showing that A* can reduce node expansions while maintaining correctness. A major contributor to perceived "wrong routes" was data quality and visualization noise rather than algorithm failure. By introducing validated production datasets, resolving ID/name mismatches, de-duplicating city names, and removing impossible road distances.

Key Achievements:

- 100+ Ethiopian cities modeled with real coordinates
- 106 validated roads with realistic distances (filtered from 500+ raw roads)
- Three algorithms implemented (Array Dijkstra, PQ Dijkstra, A*)
- Interactive web dashboard with analytics
- MySQL database for persistent storage
- Validated dataset pipeline for data quality

References

- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*.
- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*.
- Folium documentation (interactive maps).
- SQLAlchemy documentation (ORM/database sessions).

Appendix A: Key Output Files

- Validated datasets:
  - data/processed/ethiopian_cities_validated.csv
  - data/processed/ethiopia_road_network_validated.csv
- Flagged roads audit:
  - data/processed/ethiopia_road_network_flagged.csv
- Dataset generation tool:
  - scripts/regenerate_validated_routing_data.py